

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

—o0o—

BÁO CÁO DỰ ÁN

RICE MONITORING SYSTEM

Hệ thống giám sát và phân tích dữ liệu lúa thời gian thực

Sinh viên thực hiện: Võ Hữu Nghĩa
Mã số sinh viên: 3124410231
Chuyên ngành: Khoa học Máy tính

TP. HỒ CHÍ MINH, THÁNG 04/2026

Mục lục

I	KIẾN THỨC NỀN TẢNG	2
I.1	ẢNH VỆ TINH LÀ GÌ? (Raster Data)	2
I.1.1	Raster vs Vector	2
I.1.2	Cấu trúc dữ liệu của một ảnh Raster trong bộ nhớ	2
I.1.3	Sentinel-2 — Band nào dùng để tính NDVI?	2
I.1.4	Định dạng file	3
I.2	THÔNG TIN ĐỊA LÝ ĐÍNH KÈM (Geospatial Metadata)	3
I.2.1	CRS — Hệ tọa độ tham chiếu (Coordinate Reference System)	3
I.2.2	Affine Transform Matrix — Ma trận biến đổi	3
I.3	THƯ VIỆN (rasterio) — CẦU NỐI PYTHON ↔ FILE ẢNH	4
I.3.1	Rasterio là gì?	4
I.3.2	Khái niệm cốt lõi: Dataset Object	4
I.4	QUẢN LÝ BỘ NHỚ CƠ BẢN	4
I.4.1	Tính toán RAM tiêu thụ	4
II	CẤU TRÚC ĐẦY ĐỦ CỦA MA TRẬN AFFINE	5
II.1	TẠI SAO e ÂM?	5
II.2	TRUY CẬP TỪNG PHẦN TỬ CỦA TRANSFORM TRONG RASTERIO	5
II.3	PIXEL SIZE (GSD) LÀ GÌ?	6
II.4	BOUNDING BOX LÀ GÌ?	6
II.4.1	Trường hợp lý tưởng ($b, d = 0$ - Ảnh không bị xoay)	6
II.4.2	Trường hợp tổng quát ($b, d \neq 0$ - Ảnh bị xoay nghiêng)	7
II.5	ỨNG DỤNG NHÂN MA TRẬN AFFINE	7
II.6	TỌA ĐỘ TRẢ VỀ: GÓC PIXEL HAY TÂM PIXEL?	8
II.7	TỔNG KẾT LOGIC XỬ LÝ & CHECKLIST	8
III	KIẾN TRÚC DATA INGESTION & GIỚI HẠN HỆ THỐNG	10
III.1	PHÂN TÍCH CHI PHÍ I/O: STREAMING VS BATCH DOWNLOAD	10
III.1.1	Mô hình hoá chi phí Streaming	10
III.1.2	So sánh với Local SSD	10
III.2	CƠ CHẾ KÍCH HOẠT: PULL VS PUSH	11
III.3	GIỚI HẠN CỦA STAC API	12
III.3.1	Phân tích lỗ hổng	12
III.3.2	Minh chứng thực nghiệm	12
III.3.3	Known Limitations — Phase 1	13

Chương I

KIẾN THỨC NỀN TẢNG

I.1 ẢNH VỆ TINH LÀ GÌ? (Raster Data)

I.1.1 Raster vs Vector

Trong GIS (Hệ thống thông tin địa lý) có 2 kiểu dữ liệu chính:

- **Vector:** Lưu theo dạng điểm, đường, vùng (polygon). Ví dụ: ranh giới tỉnh thành.
- **Raster:** Lưu dưới dạng **lưới ô vuông (Grid)**, mỗi ô là một pixel có giá trị số. → Ảnh vệ tinh là Raster.

I.1.2 Cấu trúc dữ liệu của một ảnh Raster trong bộ nhớ

Hãy hình dung ảnh Raster là một **mảng 3 chiều (3D Array)**:

```
[Band(dải), Row(hàng), Column(cột)] <- NumPy shape
4          10980          10980 <- kích thước thực Sentinel-2
```

- **Band (Băng tần):** Mỗi band là 1 "kênh" ánh sáng khác nhau (Đỏ, Xanh, Hồng ngoại...).
- **Row × Column:** Chiều cao × Chiều rộng ảnh tính bằng số pixel.
- **Giá trị mỗi pixel:** Là số nguyên **uint16** (0 → 65535), biểu diễn cường độ phản xạ ánh sáng.

I.1.3 Sentinel-2 — Band nào dùng để tính NDVI?

Band	Tên	Bước sóng	Độ phân giải	Dùng để
B04	RED	665 nm	10m/px	Ánh sáng đỏ cây hấp thụ
B08	NIR	842 nm	10m/px	Hồng ngoại cây phản xạ

Kết luận

Chỉ cần **2 band: B04 và B08** để tính NDVI.

I.1.4 Định dạng file

- **GeoTIFF** (.tif): Định dạng phổ biến nhất, chứa cả dữ liệu pixel lẫn thông tin địa lý (CRS, Transform).
- **JPEG2000** (.jp2): Định dạng nén Sentinel-2 gốc tải về từ Copernicus Hub.

I.2 THÔNG TIN ĐỊA LÝ ĐÍNH KÈM (Geospatial Metadata)

Điều khác biệt lớn nhất của ảnh vệ tinh so với ảnh chụp thông thường là nó có **tọa độ thực địa** gắn liền. Khi mở file, bạn không chỉ đọc pixel mà còn cần đọc 2 thứ quan trọng:

I.2.1 CRS — Hệ tọa độ tham chiếu (Coordinate Reference System)

- Mô tả cách "**chiều**" bề mặt cầu Trái Đất lên mặt phẳng 2D.
- Sentinel-2 thường dùng **EPSG:32648** (UTM Zone 48N) cho khu vực Việt Nam.
- Được biểu diễn bằng chuỗi **WKT** hoặc mã **EPSG**.

Ví dụ

"EPSG:32648" → UTM Zone 48N → Phù hợp Đồng bằng sông Cửu Long.

I.2.2 Affine Transform Matrix — Ma trận biến đổi

Đây là ma trận 3x3 ánh xạ từ tọa độ pixel (col, row) sang **tọa độ phẳng (Planar Coordinates - X, Y)**. Để chuyển tiếp từ không gian phẳng này sang **tọa độ địa lý mặt cầu (Kinh độ, Vĩ độ)**, dữ liệu cần được đưa qua các phương trình biến đổi của hệ quy chiếu (CRS).

$$\begin{pmatrix} x_{\text{real}} \\ y_{\text{real}} \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} col \\ row \\ 1 \end{pmatrix}$$

Trong đó:

- **(c, f)**: Tọa độ thực địa của góc trên-trái của ảnh (origin).
- **a**: Kích thước pixel theo chiều X (thường = +10.0 mét với Sentinel-2 10m).
- **e**: Kích thước pixel theo chiều Y (thường = **-10.0** vì trục Y đảo ngược: hàng 0 ở trên).
- **b, d**: Thường = 0 nếu ảnh không bị xoay.

Tại sao cần matrix này?

Để sau khi tính NDVI xong, có thể ghi kết quả ra file mới mà vẫn giữ nguyên tọa độ thực địa.

I.3 THƯ VIỆN (rasterio) — CẦU NỐI PYTHON ↔ FILE ẢNH

I.3.1 Rasterio là gì?

rasterio là thư viện Python wrapper của **GDAL** (Geospatial Data Abstraction Library - viết bằng C++). Nó giúp Python đọc/ghi các định dạng GeoTIFF, JP2, HDF5... mà không cần tự parse binary file.

I.3.2 Khái niệm cốt lõi: Dataset Object

Khi mở file với rasterio, nhận về một **Dataset Object**.

```
dataset = rasterio.open("data/01_raw/T48PXS_B04.jp2")

# Metadata
dataset.count      # Band number (usually = 1 for jp2)
dataset.width      # Column number (pixel)
dataset.height     # Row number (pixel)
dataset.dtypes     # Data type ('uint16',)
dataset.crs        # Coordinate Reference System
dataset.transform   # Affine Transform Matrix

data = dataset.read(1) # shape: (height, width)

dataset.close()
```

I.4 QUẢN LÝ BỘ NHỚ CƠ BẢN

I.4.1 Tính toán RAM tiêu thụ

```
1 pixel Sentinel-2 = uint16 = 2 bytes
1 band B04         = 10980 * 10980 * 2 bytes
                  = 241,120,800 bytes
                  ~ 230 MB (cho 1 band)

Cả 4 band (B02,B03,B04,B08) = ~920 MB
```

Chương II

CẤU TRÚC ĐẦY ĐỦ CỦA MA TRẬN AFFINE

$$\begin{pmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{pmatrix}$$

Ký hiệu	Tên gọi	Giá trị trong ảnh test	Ý nghĩa
c	X-origin	560000.0	Tọa độ X của góc trên-trái
f	Y-origin	1075120.0	Tọa độ Y của góc trên-trái
a	X pixel size	+10.0	Mỗi bước sang phải 1 cột = thêm 10 mét
e	Y pixel size	-10.0	Mỗi bước xuống 1 hàng = giảm 10 mét
b, d	Rotation	0.0	Ảnh không bị xoay

II.1 TẠI SAO e ÂM?

Đây là điểm gây nhầm lẫn nhiều nhất với người mới:

Hệ tọa độ MẢNG (NumPy):

Row 0 <- Hàng ĐẦU TIÊN

Row 1

Row 2

...

Row N <- Hàng CUỐI CÙNG

Hệ tọa độ BẢN ĐỒ (thực địa):

Y tăng lên = về phía Bắc (Lên)

Y giảm xuống = về phía Nam (Xuống)

Row 0 trong mảng = Vị trí **phía Bắc nhất** trên bản đồ (Y lớn nhất).
 Row 1 = bước xuống 1 hàng → đi về phía Nam → Y **giảm**.

Kết luận

$e = -10.0$ vì “đi xuống 1 hàng pixel = giảm 10 mét tọa độ Y”.

II.2 TRUY CẬP TỪNG PHẦN TỬ CỦA TRANSFORM TRONG RASTERIO

```

t = dataset.transform      # object affine.Affine

t.a    # pixel size theo X (thường = +10.0)
t.b    # rotation x (thường = 0.0)
t.c    # X tọa độ góc trên-trái
t.d    # rotation y (thường = 0.0)
t.e    # pixel size theo Y (thường = -10.0)
t.f    # Y tọa độ góc trên-trái

```

II.3 PIXEL SIZE (GSD) LÀ GÌ?

Pixel size (hay Ground Sampling Distance — GSD) là **kích thước thực địa của 1 pixel** trên mặt đất.

Sentinel-2 Band 4 và Band 8: mỗi pixel = 10m × 10m
(1 pixel đại diện cho 10m × 10m diện tích thực)

Đọc pixel size từ đâu?

Pixel size nằm ngay trong ma trận Affine:

- **Chiều X** → lấy giá trị tuyệt đối của `transform.a` → $|t.a| = 10.0$
- **Chiều Y** → lấy giá trị tuyệt đối của `transform.e` → $|t.e| = 10.0$

Vì sao dùng `abs()`?

Vì `e` âm nhưng kích thước pixel luôn là khoảng cách vật lý (số dương).

II.4 BOUNDING BOX LÀ GÌ?

Bounding Box (hộp giới hạn) là hình chữ nhật **nhỏ nhất** bao quanh toàn bộ ảnh, được mô tả bằng 4 tọa độ cực đại thực địa:

```

      <- left <-                                -> right ->
      |                                           |
TOP (North)  +-----+-----+ ^ top
      |                                           |
              TOÀN BỘ ẢNH
      |                                           |
BOTTOM(South) +-----+-----+ v bottom

```

II.4.1 Trường hợp lý tưởng ($b, d = 0$ - Ảnh không bị xoay)

Hệ phương trình tọa độ:

$$x = a \cdot col + c$$

$$y = e \cdot row + f$$

Để tìm giới hạn, ta xét các biên của ma trận ảnh:

- Để x_{min} thì $col = 0 \rightarrow left = c$
- Để x_{max} thì $col = width \rightarrow right = c + a \cdot width$
- Để y_{max} thì $row = 0 \rightarrow top = f$
- Để y_{min} thì $row = height \rightarrow bottom = f + e \cdot height$

Ví dụ minh họa:

$$\begin{aligned} left &= 560000.0 \\ right &= 560000.0 + 10.0 \times 512 = 565120.0 \\ top &= 1075120.0 \\ bottom &= 1075120.0 + (-10.0) \times 512 = 1070000.0 \end{aligned}$$

II.4.2 Trường hợp tổng quát ($b, d \neq 0$ - Ảnh bị xoay nghiêng)

Hệ phương trình tọa độ đầy đủ:

$$\begin{aligned} x &= a \cdot col + b \cdot row + c \\ y &= d \cdot col + e \cdot row + f \end{aligned}$$

Xác định tọa độ (x, y) tại 4 góc của ma trận ảnh:

- Góc trên-trái ($col = 0, row = 0$): $x_1 = c, y_1 = f$
- Góc trên-phải ($col = width, row = 0$): $x_2 = a \cdot width + c, y_2 = d \cdot width + f$
- Góc dưới-trái ($col = 0, row = height$): $x_3 = b \cdot height + c, y_3 = e \cdot height + f$
- Góc dưới-phải ($col = width, row = height$): $x_4 = a \cdot width + b \cdot height + c, y_4 = d \cdot width + e \cdot height + f$

Tọa độ hộp giới hạn (Bounding Box) chứa toàn bộ ảnh xoay:

$$\begin{aligned} left &= \min(x_1, x_2, x_3, x_4) \\ right &= \max(x_1, x_2, x_3, x_4) \\ top &= \max(y_1, y_2, y_3, y_4) \\ bottom &= \min(y_1, y_2, y_3, y_4) \end{aligned}$$

II.5 ỨNG DỤNG NHÂN MA TRẬN AFFINE

Đây là ứng dụng trực tiếp của phép nhân ma trận. Phương trình tổng quát chuyển đổi tọa độ điểm ảnh sang tọa độ thực địa:

$$\begin{pmatrix} x_{\text{real}} \\ y_{\text{real}} \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} col \\ row \\ 1 \end{pmatrix}$$

Khai triển hệ phương trình (khi ảnh không bị xoay, tức là $b = 0, d = 0$):

$$\begin{aligned} x_{\text{real}} &= a \cdot col + c \\ y_{\text{real}} &= e \cdot row + f \end{aligned}$$

II.6 TỌA ĐỘ TRẢ VỀ: GÓC PIXEL HAY TÂM PIXEL?

Trong quá trình xử lý hệ thống thông tin địa lý (GIS), cần phân biệt rõ 2 quy ước tính toán tọa độ điểm ảnh:

- **Góc trên-trái của pixel:** Rasterio mặc định sử dụng chuẩn này. Ví dụ với góc tọa độ `'(row=0, col=0)'` $\rightarrow$ `'(560000, 1075120)'`.
- **Tâm pixel (Center Pixel):** Tọa độ bị dịch chuyển (offset) thêm một nửa độ lớn của pixel size theo mỗi trục. Ví dụ `'(row=0, col=0)'` $\rightarrow$ `'(560005, 1075115)'`.

Lưu ý triển khai

Trong dự án thực tế của báo cáo này, hệ thống đã được can thiệp để implement logic lấy **tâm pixel** nhằm tăng độ chính xác khi đối chiếu với dữ liệu GPS (tọa độ GPS trên thực địa mặc định được coi là tâm của một vùng không gian).

II.7 TỔNG KẾT LOGIC XỬ LÝ & CHECKLIST

Checklist xác thực kiến thức

- ✓ Hiểu rõ `transform.a` = pixel size theo trục X và `transform.e` = pixel size theo trục Y (thường là số âm).
- ✓ Nắm vững lý do `e` mang giá trị âm: Trong hệ tọa độ mảng, chỉ số hàng (row) tăng dần từ trên xuống dưới, tương ứng với việc tọa độ Y trên thực địa đang giảm dần (đi về phía Nam).
- ✓ Làm chủ phương trình ánh xạ tọa độ: $x = a \cdot col + c$ và $y = e \cdot row + f$.
- ✓ Tự tính toán được Bounding Box (khung bao giới hạn) từ tập hợp tham số `c`, `f`, `a`, `e`, `width`, `height`.
- ✓ Nắm được hành vi mặc định của `pixel_to_coords(0,0)` là trả về tọa độ ở góc trên-trái (top-left).

Bảng ánh xạ hàm xử lý

Hàm xử lý	Nguồn dữ liệu đầu vào	Công thức / Logic tính toán
<code>get_pixel_size()</code>	<code>transform.a</code> , <code>transform.e</code>	Lấy trị tuyệt đối: $(\text{abs}(a), \text{abs}(e))$
<code>get_bounds()</code>	<code>transform.c/f/a/e</code> & W/H	Biên phải: $\text{right} = c + a \cdot W$ Biên dưới: $\text{bottom} = f + e \cdot H$
<code>pixel_to_coords()</code>	<code>transform.a/c/e/f</code>	$x = a \cdot \text{col} + c$ $y = e \cdot \text{row} + f$ (*Cộng thêm $\text{offset } \frac{a}{2}, \frac{e}{2}$ nếu dùng tâm pixel)

Chương III

KIẾN TRÚC DATA INGESTION & GIỚI HẠN HỆ THỐNG

III.1 PHÂN TÍCH CHI PHÍ I/O: STREAMING VS BATCH DOWNLOAD

Khi thiết kế tầng thu nạp dữ liệu (Data Ingestion), có hai chiến lược truy cập ảnh vệ tinh lưu trữ dưới dạng Cloud Optimized GeoTIFF (COG):

- **Level 1 — Batch Download:** Tải toàn bộ file COG về `data/01_raw/` qua STAC API, sau đó đọc cục bộ bằng `read_chunk()` (windowed reading, xem Chương I).
- **Level 2 — Cloud-Native Streaming:** Dùng hệ thống file ảo `/vsicurl/` của GDAL, mỗi lệnh gọi `read(window=...)` phát sinh một **HTTP Range Request** độc lập tới server lưu trữ.

III.1.1 Mô hình hoá chi phí Streaming

Với một tile Sentinel-2 kích thước 10980×10980 pixel, chia thành các chunk vuông cạnh $c = 256$ pixel, số lượng chunk cần đọc để quét hết một band là:

$$N_{chunks} = \left\lceil \frac{W}{c} \right\rceil \times \left\lceil \frac{H}{c} \right\rceil = \left\lceil \frac{10980}{256} \right\rceil^2 = 43^2 = 1849 \quad (\text{III.1})$$

Mỗi HTTP Range Request qua `/vsicurl/` chịu một độ trễ round-trip cố định t_{lat} (đọc lập với kích thước dữ liệu tải về, do bản chất TCP handshake + TLS negotiation + HTTP header overhead), ước lượng bảo thủ $t_{lat} \approx 100\text{ms}$. Với chiến lược **exhaustive full-tile scan** (đọc toàn bộ ảnh, không chỉ một vùng nhỏ), tổng độ trễ mạng tối thiểu là:

$$T_{stream} = N_{chunks} \times t_{lat} = 1849 \times 0.1\text{s} \approx 185\text{s} \quad (\text{III.2})$$

III.1.2 So sánh với Local SSD

Theo tính toán bộ nhớ ở Chương I (Mục “Quản lý bộ nhớ cơ bản”), một band uint16 kích thước đầy đủ chiếm $S_{band} \approx 230\text{MB}$. Với thông lượng đọc tuần tự bảo thủ của SSD hiện đại $v_{SSD} \approx 500\text{MB/s}$ (thấp hơn nhiều so với NVMe cao cấp $\sim 3500\text{MB/s}$, chọn giá trị thấp để so sánh công bằng):

$$T_{local} = \frac{S_{band}}{v_{SSD}} = \frac{230\text{MB}}{500\text{MB/s}} \approx 0.46\text{s} \quad (\text{III.3})$$

Chiến lược	Thời gian ước lượng	Bottleneck
Streaming (/vsicurl/, 1849 requests)	$\approx 185s$	Network round-trip latency
Batch Download + Local Read	$\approx 0.46s$ (sau khi đã tải)	Disk throughput

Bảng III.1: So sánh chi phí I/O giữa hai chiến lược cho một lần exhaustive full-tile scan.

Kết luận thiết kế

Chênh lệch xấp xỉ **400×** không đến từ băng thông (bandwidth) mà từ **độ trễ trên mỗi request** (latency-bound vs throughput-bound). Streaming chỉ tối ưu khi khối lượng dữ liệu thực sự đọc nhỏ hơn khối lượng sẽ phải tải về (truy vấn một AOI nhỏ trong một catalog lớn). Với bài toán giám sát lúa cấp tỉnh — vốn đòi hỏi quét toàn bộ diện tích mỗi tháng — workload là *exhaustive*, nên **Level 1 (Batch Download)** là **lựa chọn đúng**, không phải bước đệm tạm thời chờ “nâng cấp” lên Level 2.

III.2 CƠ CHẾ KÍCH HOẠT: PULL VS PUSH

Bên cạnh chiến lược truy cập dữ liệu, hệ thống ingestion cần một cơ chế kích hoạt (trigger):

- **Pull:** Hệ thống chủ động truy vấn STAC API theo lịch cố định (cronjob).
- **Push:** Hệ thống lắng nghe sự kiện thông báo (ví dụ AWS SNS publish khi có scene mới), phản ứng gần như tức thời.

Nguyên lý lựa chọn

Lựa chọn đúng phụ thuộc vào **hằng số thời gian (time constant)** của hiện tượng vật lý cần quan sát, không phải vào khả năng kỹ thuật sẵn có.

Chu kỳ sinh trưởng của cây lúa trải qua các giai đoạn sinh lý rõ rệt:

Giai đoạn sinh trưởng	Thời gian (τ_{physio})
Mạ (Seedling)	$\sim 15\text{--}20$ ngày
Đẻ nhánh (Tillering)	$\sim 25\text{--}30$ ngày
Làm đòng (Booting)	$\sim 25\text{--}30$ ngày
Trổ bông (Flowering)	~ 10 ngày
Chín (Ripening)	~ 30 ngày

Bảng III.2: Các giai đoạn sinh trưởng của lúa và thời gian đặc trưng.

Biến động NDVI có ý nghĩa thống kê chỉ xuất hiện ở quy mô **tuần**, không phải giờ hay phút. Chu kỳ revisit của vệ tinh Sentinel-2 (~ 5 ngày) đã cung cấp độ phân giải thời gian dư thừa so với tốc độ biến đổi sinh lý này. Điều kiện để Push có giá trị thực sự là:

$$\tau_{physio} \lesssim \tau_{poll} \quad (\text{III.4})$$

tức thời gian giữa hai lần poll phải *chậm hơn* tốc độ biến đổi của hiện tượng thì mới bỏ lỡ thông tin. Trong bài toán này, $\tau_{poll} = 1$ tháng $\ll \tau_{physio}$ (một chu kỳ sinh trưởng đầy đủ $\sim 3-4$ tháng), nên Pull với tần suất hàng tháng là **dư thừa an toàn**, không thiếu hụt thông tin.

Ngược lại, Push đòi hỏi duy trì một tiến trình lắng nghe thường trực (server standby, subscription tới message broker) — chi phí vận hành liên tục để đổi lấy độ trễ phản ứng thấp hơn nhiều bậc so với mức cần thiết của hiện tượng quan sát.

Kết luận thiết kế

Dầu tư responsiveness vượt quá tốc độ thay đổi của đối tượng được giám sát không tạo thêm giá trị, chỉ tạo thêm chi phí vận hành (over-engineering). **Kiến trúc Pull (cronjob hàng tháng) được chọn cho hệ thống hiện tại.**

III.3 GIỚI HẠN CỦA STAC API

III.3.1 Phân tích lỗ hổng

Đặc tả STAC (SpatioTemporal Asset Catalog) **không quy định** thứ tự sắp xếp mặc định của kết quả truy vấn — đây là hành vi phụ thuộc vào từng implementation backend cụ thể, không phải một hợp đồng (contract) được đảm bảo. Với tham số `max_items=1` kết hợp một bbox giao cắt với nhiều tile UTM lân cận, truy vấn không trả về tập hợp các tile phủ kín bbox, mà chỉ trả về **một phần tử đơn lẻ theo tiêu chí sort ngầm định** của backend:

$$\text{result} = \arg \text{sort}_{\text{criterion}} \left(\{i \in I : \text{footprint}(i) \cap \text{bbox}_{\text{query}} \neq \emptyset\} \right) [0] \quad (\text{III.5})$$

trong khi truy vấn *đúng* về mặt phủ không gian đầy đủ phải là hợp toàn bộ tập giao cắt:

$$\text{Items}_{\text{correct}} = \{i \in I : \text{footprint}(i) \cap \text{bbox}_{\text{query}} \neq \emptyset\} \quad (\text{III.6})$$

III.3.2 Minh chứng thực nghiệm

Truy vấn trực tiếp Earth Search STAC API v1 với `bbox = [105.0, 9.5, 106.0, 10.5]` (khu vực Đồng bằng sông Cửu Long) trả về đồng thời hai scene thuộc hai tile UTM khác nhau:

S2B_48PVS_20230115_0_L2A	cloud_cover = 17.07%
S2B_48PWR_20230115_0_L2A	cloud_cover = 17.94%

Với `max_items=1`, một trong hai tile này bị loại bỏ khỏi kết quả một cách âm thầm — không phát sinh lỗi (exception), không có cảnh báo. Hệ quả: nếu ranh giới hành chính một tỉnh trải dài qua nhiều tile, phần diện tích nằm ngoài tile được chọn sẽ hoàn toàn vắng mặt trong thống kê, trong khi hệ thống vẫn báo cáo thành công.

III.3.3 Known Limitations — Phase 1

Giới hạn đã biết của Phase 1

- ✗ **Multi-tile Mosaicking** chưa được hỗ trợ. Tầng ingestion hiện tại (`search_and_download`) giả định bbox đầu vào nằm gọn trong một tile UTM duy nhất.
- ✗ Chưa xử lý **CRS harmonization** khi bbox trải qua ranh giới hai UTM zone (ví dụ 48N/49N).
- ✗ Chưa có bước **overlap deduplication** cho vùng chồng lấn giữa các tile liên kề có thời điểm chụp khác nhau.

Quyết định thiết kế: hoãn (defer) bài toán mosaicking sang một lớp orchestrator riêng, xây dựng *sau* khi lõi tính toán NDVI (Chương tiếp theo) đã ổn định trên đơn vị xử lý cơ bản — một tile. Lý do: mosaicking là bài toán **spatial geometry** (hợp nhất, khử trùng lặp không gian), tách biệt hoàn toàn về bản chất so với bài toán **matrix computation** (vector hoá phép tính NDVI trên mảng NumPy) của lõi xử lý. Trộn lẫn hai mối quan tâm này ngay từ đầu sẽ làm tăng độ phức tạp debug khi xảy ra sự cố tràn bộ nhớ (OOM).